

Project Report: Classification of Fresh and Rotten Fruits Using PyTorch

1. Introduction and Overview

This project aims to apply the knowledge acquired during the PyTorch course to solve different challenges in image analysis—starting with basic tasks like classification and progressing toward more complex, real-world applications.

In the first stage, the Fruit Image Dataset was used with both Fresh and Rotten Fruits, which contains images divided into six categories: apples, bananas, and oranges — both fresh and rotten. The dataset is organized into separate folders for training and testing. Using this dataset, we trained a convolutional neural network (CNN) from scratch to build a model capable of distinguishing between the different fruit categories. This stage involved data preprocessing, model design, training, evaluation, and presenting the results.

To deepen the analysis and apply more advanced techniques from the course, we later used a pretrained EfficientNet model, which was fine-tuned to classify the same six fruit categories. We also added functionality to analyze external images and used Grad-CAM — a visualization tool that helps interpret which parts of the image the model focused on when making predictions.

To take the project even further, we implemented a DETR (DEtection TRansformer) model, supported by a ResNet-50 backbone and pretrained on the COCO dataset. This model allowed us not only to recognize objects but also to locate them within images.

Building on this, we applied DETR to video inputs, enabling object detection in motion — an important step toward more dynamic and practical real-world applications.

Finally, we explored a potential real-world use case: integrating these models into computer vision systems for machines or robots. This would allow them to perceive and understand their surroundings in real time. Such visual analysis could be applied in areas like automation, agriculture, manufacturing, and beyond.

2. Project Evolution

We initially began with the implementation of a CNN model for fruit image classification. This model performed well on both the test dataset and external images. Subsequently, to achieve a deeper level of analysis, we incorporated a pretrained EfficientNet model, fine-tuned for our specific classification task.

Later, we shifted our focus from image classification to object detection, implementing the DETR model for this purpose. Finally, we extended this implementation by applying DETR to video input, enabling the detection of elements in real-time sequences.

3. Code Implementations

Inside the Report Details folder, you'll find the detailed implementations of each model used:

Report-A: Contains the initial CNN implementation trained with the Fruit Image Dataset: Fresh and Rotten Fruits.

Report-B: Includes the implementation of a pretrained EfficientNet model adapted to classify the six fruit categories. Additionally, Grad-CAM was integrated to visualize where the model focuses during classification.

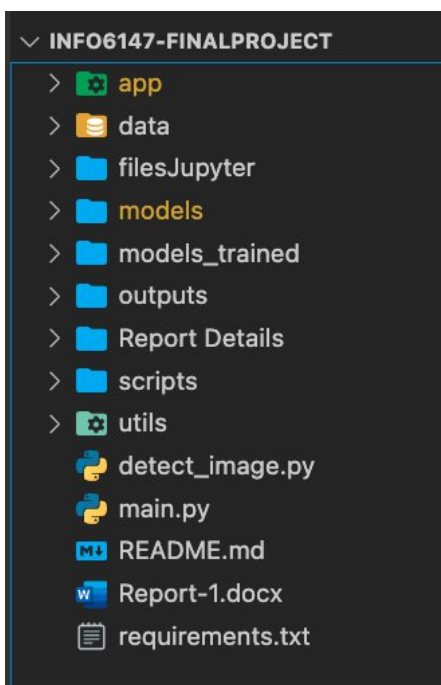
Report-C: Contains the DETR model implementation, adapted from the class example and modified to meet our objective of fruit detection.

Report-D: Builds on the previous implementation by applying DETR to video inputs for real-time object detection.

4. Project Development

Once the base implementations were completed individually, the next goal was to integrate them into a web-based project to provide more realistic and accessible interaction with the models.

The main components of the project are structured as follows:



-app/: Contains the web interface and server code required to deploy the application.

-data/: Stores the datasets used for training and evaluation.

-models/: Includes the saved model files used in the application.

-utils/: Contains utility functions for connecting the models with the web server.

This modular structure allows for scalability and the incorporation of additional models and features as needed. It also helps organize and deploy the implemented models in a clean and maintainable way.

5. Real-World Application

The DETR model is a strong candidate for object detection and segmentation tasks, making it a valuable tool for machine and robotic vision. By leveraging transformer architecture, DETR overcomes the limitations of traditional convolution-based detection methods, offering improved generalization and flexibility in complex scenarios.

In robotic applications, this model could enable robots to accurately recognize and locate objects within their environment, facilitating tasks such as automated manipulation, navigation in dynamic spaces, and human interaction. Additionally, DETR's scalability allows it to adapt to various sensors and environments, improving the autonomy and efficiency of robotic systems across industrial, logistical, and service domains.

6. Challenges Encountered

The transition from an academic approach using Jupyter Notebook to a more professional, web-oriented project posed a significant challenge. It required a deep understanding of the models themselves and also good code structuring to ensure a functional and maintainable final product.

While image classification and detection models share some foundational concepts, each presents its own challenges. As model efficiency increases, so does complexity, requiring more in-depth technical knowledge for correct implementation, and interpretation.

7. References

-Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-End object detection with transformers (DETR). European Conference on Computer Vision (ECCV).

<https://arxiv.org/abs/2005.12872>

-Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32.

https://papers.nips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

-Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. International Conference on Machine Learning (ICML). <https://arxiv.org/abs/1905.11946>

-Torchvision. (n.d.). Torchvision library – PyTorch. PyTorch. <https://pytorch.org/vision/stable/index.html>

-Kritik Seth. (n.d.). Fruit Image Dataset: Fresh and Rotten Fruits. Kaggle.

<https://www.kaggle.com/datasets/kritikseth/fruit-image-dataset>